

University of Arizona
CSC483 Text Retrieval
Spring 2026
Prof. Mihai Surdeanu
DUE: MAY 6TH

Final Project — Baby Watson
Jessie McManus, Annabelle Jonatan
(Undergraduate Students)

QUESTIONS

See results (written) for 1, 2, and results (program) for 3, 4.

1: CORE IMPLEMENTATION

- Indexing and retrieval (Lucene or Whoosh?)
- What specific problems did we have with Wikipedia content?
- How did you build the query from the clue? (discuss to/to not concatenate?)

2: CODING WITH LLMS

- What LLMs did you use?
- How did you prompt them?
- How did you use the LLM (vibe code! also debug)?
- What worked and what did not?

3: MEASURING PERFORMANCE

-Measure using metrics discussed in class (precision at 1 $P@1$, normalized discounted cumulative gain NDCG, mean reciprocal rank MRR). Justify choice, report performance

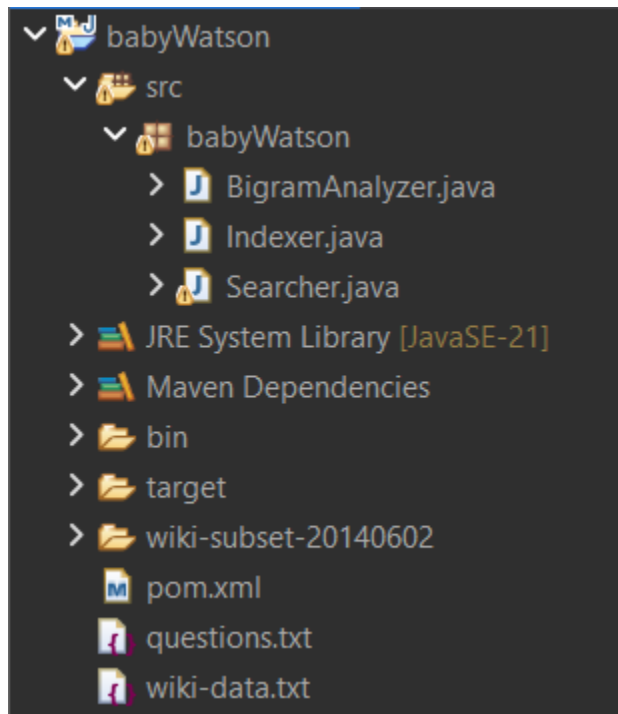
4: ERROR ANALYSIS

- How many correct/incorrect answers?
- Why do you think a simple system can answer those questions?
- What problem did you observe for the incorrect questions? (group errors into classes)

REQUIRED PORTIONS

HOW TO COMPILE

Download babyWatson.zip, unzip and import into Eclipse. Access all wiki documents from the wiki-data.txt by unpackaging the tar and moving the unpackaged wiki subset folder into the same directory. Run first Indexer.java (which requires and references BigramAnalyzer.java) until it prints a confirmation message, then upon the construction of the index, run Searcher.java. View results in the text window.



DESCRIPTION OF CODE

“Indexer.java”: Indexes the Wikipedia articles provided with a combination of bigram and unigram indexing across each entire document. Does not attempt to parse any Wikipedia-specific formatting beyond starting a new document upon detection of the next title (ie. “[[TITLE]]”) and for handling redirect links.

“BigramAnalyzer.java”: What is said on the tin; extends the Analyzer class to add a Bigram index analyzer.

“Searcher.java”: Reads in questions.txt and parses out the clue from each of the 100 questions, discarding categories and answers, and then processing each individual clue as a search query and

returning the top document from each - null if none found. This searcher uses BM25, bigrams and unigrams. Finally, the searcher prints out the number of successful searches and the number of nulls.

RESULTS (FROM PROGRAM) (see: questions)

MEASURING PERFORMANCE: Our combined Indexer and Searcher produce a P@1 of 17%. Our reasoning for choosing P@1 is that all other forms of measurement were inapplicable. NDCG is best when an answer can be *somewhat* correct, and there is only one correct document for each clue. Furthermore, MAP is good when there are multiple correct answers, which is untrue for Jeopardy. MRR would have been good if we were trying to answer within a certain number of answers (top 5 answers, or a search engine product). Due to our Watson model needing the singular top answer to play jeopardy, we figured it made little sense to use MRR to grade our system's accuracy.

ERROR ANALYSIS: As implied prior, we have 17 correctly predicted documents and 83 incorrect. For why a system like ours could occasionally respond correctly, of the Jeopardy questions supplied to us were clues that could easily be found piecemeal within a Wikipedia article - for example, words in the clues might show up in the article titles. In terms of incorrect answer results, some common errors we found were: 1. Two or more potential answers would be logically similar in several regards (eg. two actors that both played The Joker), 2. Sense of a word may be ignored (eg. a person's nation of origin may be confused for information about the nation itself, returning the person and not a nation's historical event), 3. Nothing but the object "category" appears to be processed from the clue (eg. many questions asking about famous people returned other human names, just not the one we're looking for).

RESULTS (FROM WRITTEN) (see: questions)

CORE IMPLEMENTATION: We attempted first to use Whoosh but hit a wall in progressing our P@1 to anything past 10%, and so after a lot of attempted alterations (prepending category to query $\Rightarrow$ 7%, bigramming $\Rightarrow$ 3%, only lead section $\Rightarrow$ 3%), we swapped to Lucene. Whoosh appears to handle large amounts of input data poorly, being extremely slow with both indexing and searching, while Lucene is extremely fast but was far less plug-and-play to get machines to run Lucene code. We had no issues with the Wikipedia content, though we did of course have to watch for formatting that implied the beginning of a new page. When it comes to developing our search query, we had prior attempted to concatenate our category to our clue, discarding it if the category was useless (eg. "miscellaneous"), but found that that significantly damaged our P@1, and so have chosen to search with the whole clue alone.

CODING WITH LLMs: Vibe-coding the entire programming portion of this project and using it for various debuggings, we used Microsoft Copilot as this LLM is freely accessible, and Jessie has had prior familiarity with wrangling it. For initial prompts, Jessie provided snippets of the Project PDF's description of the required tasks alongside input file formatting. We found that, while Copilot is extremely quick to understand both Whoosh and Lucene for initial program writing, it has the tendency to label its solutions for improving the precision as "the most important", overestimating the usefulness of various tweaks which consistently had neutral or negative effects. When questioned why its helpful tips decreased the performance, it would often produce text claiming that *of course, <x> solution can't be used alone*, then adding more tweaks to similar deleterious effects. Vibe-coding with Copilot made a decent jump-off pad to start our assignment, but definitely needed a lot of tweaking to customize its work for our specific needs.